

ASSIGNMENT REPORT-2

Submitted by :

Mohamed Fazil [26120142]

Pranav [26120090]

Under the guidance of :

HEMA ARUNACHALAM MURTHY

For The Subject

**Pattern Recognition and Machine Learning
(26AI5707)**



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SHIV NADAR UNIVERSITY

KALAVAKAM

AUGUST 2026

1. INTRODUCTION :

This assignment is about breaking down an image into eigenvalues/eigenvectors and singular values, and then rebuilding it back using only a few of them. The point is to see how much of an image you actually need to keep to still recognise it, and to compare two ways of doing that: Eigenvalue Decomposition (EVD) and Singular Value Decomposition (SVD).

I worked with two images of my cat - one square (512x512) for the EVD + SVD part, and one rectangular (600x450) for the SVD-only part, since EVD only works on square matrices. Both were converted to grayscale first because the decomposition formulas need a single 2D matrix

2. EXPERIMENT STEPS :

Step 1: Load image and convert to grayscale

- Read the image and applied $\text{gray} = 0.299*R + 0.587*G + 0.114*B$.



Fig 1: Grayscale square image (512x512) after Step 1

Step 2 — Perform EVD: $A = Q \Lambda Q^{-1}$

- Used `np.linalg.eig(A)` to get the eigenvalues and `Q` (eigenvectors as columns).
- Took `Q_inverse` using `np.linalg.inv(Q)`
- Checked that $Q * \Lambda * Q_{\text{inverse}}$ gives back the original `A` when using all eigenvalues.

Step 3: Choose `k`, and handle complex eigenvalues

- Picked three `k` values: 10, 50 and 150
- Since the image matrix is not symmetric, some eigenvalues come out complex, always in conjugate pairs ($a+bi$ and $a-bi$).
- If only one eigenvalue of a pair is kept, the reconstructed image comes out with leftover imaginary values, which is wrong for a pixel.
- Fixed this by checking: whenever a complex eigenvalue is picked, its exact conjugate partner is pulled in too.

Step 4: Reconstruct with EVD - $A_k = Q \Lambda_k Q^{-1}$:

- Set all eigenvalues except the chosen k to zero, then multiplied $Q * \Lambda_k * Q^{-1}$.
- Took the real part only, since after keeping full conjugate pairs the imaginary part is just floating point noise.

Step 5: Compute SVD from EVD - $A = U \Sigma V^T$:

- Found V from eigenvectors of $A^T A$, and U from eigenvectors of $A A^T$.
- Singular values = square root of the eigenvalues of $A^T A$, clipped at zero first to avoid a negative number under the square root.
- $\text{eig}()$ gives eigenvectors with a random sign, so U 's sign was fixed to match $A^T v = \sigma u$ before using it.

Step 6: Reconstruct with SVD - $A_k = U \Sigma_k V^T$:

- Kept only the first k columns of U , first k singular values, first k columns of V , and multiplied them back together.
- No pairing issue here unlike EVD, since singular values are always real and already sorted biggest to smallest.

Step 7: Original, reconstruction and error image :

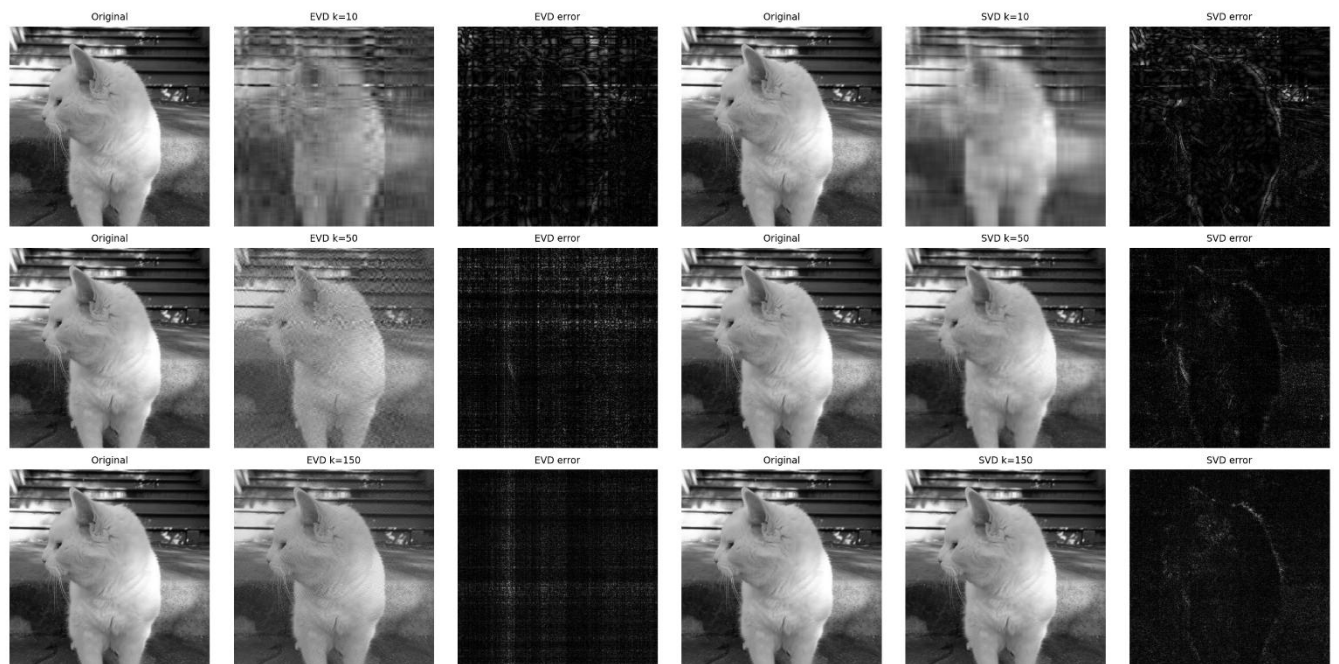


Fig 2: Each row is one k value. Left half is EVD (original, reconstruction, error), right half is SVD (original, reconstruction, error).

- At $k=10$, EVD reconstruction is barely recognisable and striped. SVD at the same k already shows a rough cat shape.
- At $k=150$, EVD is closer but still a bit patchy. SVD looks nearly identical to the original.
- EVD's error image still shows a visible grid pattern even at $k=150$. SVD's error image is close to flat black

Step 8: Frobenius error - $E(k) = \|A - A_k\|_F$

- Computed by subtracting A_k from A , squaring every value, summing them, then taking the square root.

k	EVD $E(k)$	SVD $E(k)$
10	11649.56	6824.41
50	8128.10	2385.82
150	4138.29	809.94

Step 9: Error vs k , all k from 1 to 512

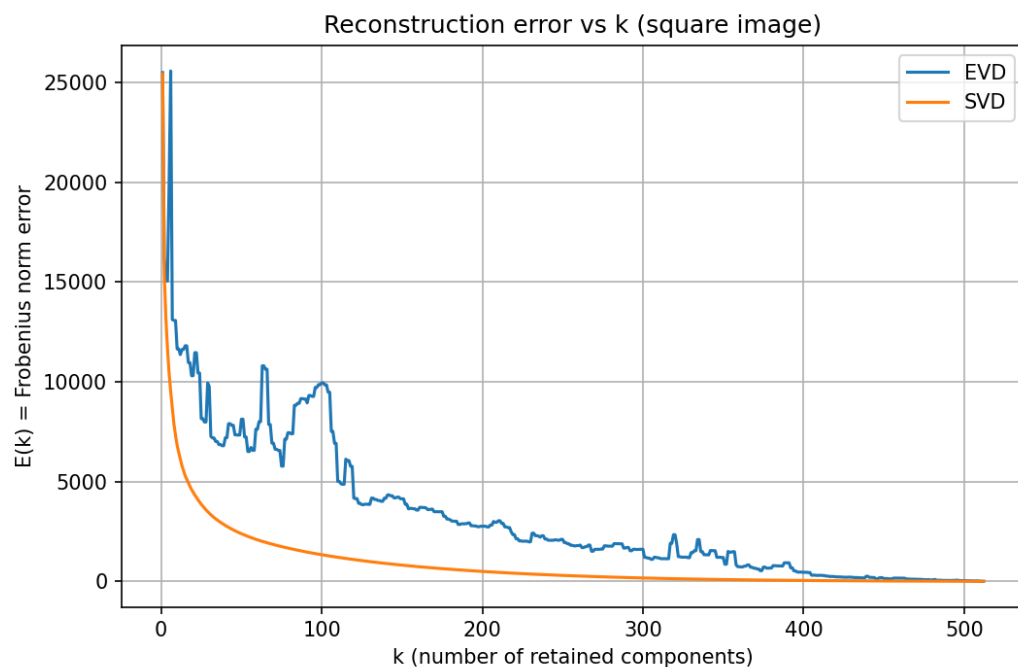


Fig 3: $E(k)$ for every k , EVD (blue) and SVD (orange)

- SVD curve drops off smoothly and flattens near zero well before $k=512$.
- EVD curve is bumpy - error goes up for a few k values before dropping again, for example around $k=60-100$. This happens consistently, not just once.

Step 10: Rectangular image - SVD only

- Same Step 1 to Step 6 as above, but only SVD, since EVD does not work on a non-square matrix.



Fig 4: Grayscale rectangular image (600x450)

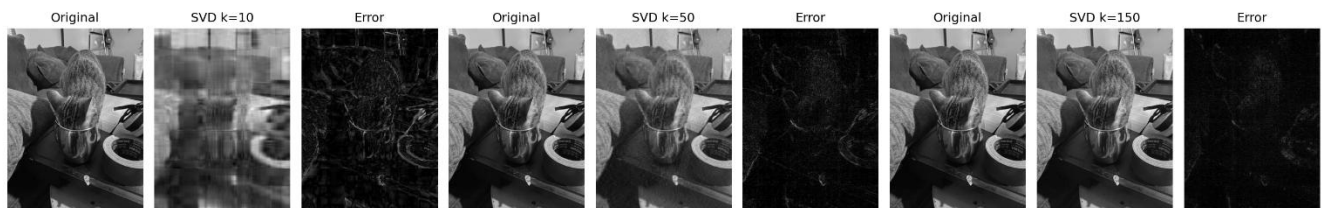


Fig 5: SVD reconstruction and error image at $k=10, 50, 150$

k	SVD $E(k)$
10	10397.45
50	4777.32
150	1688.02

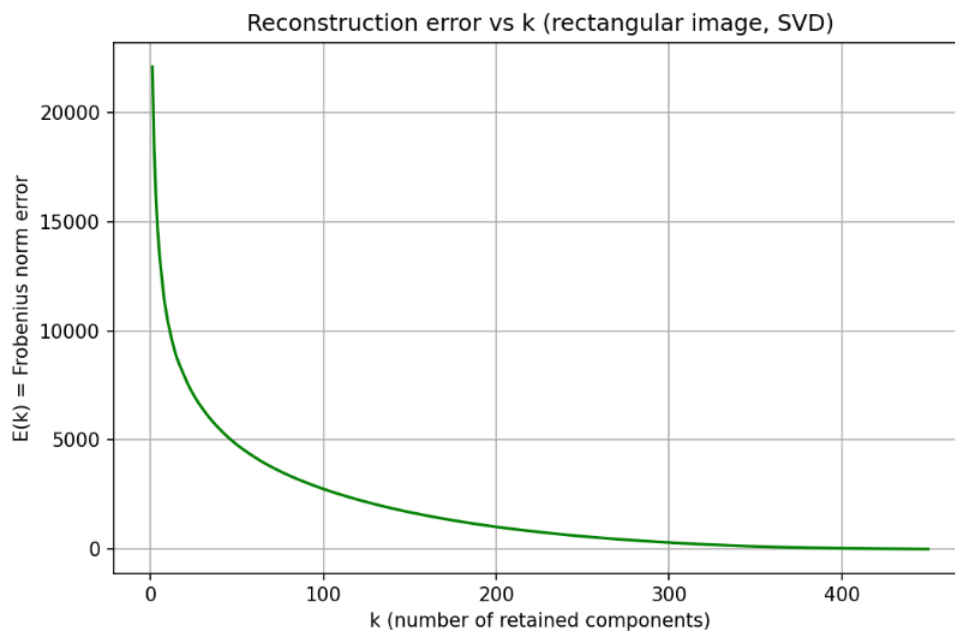


Fig 6: $E(k)$ vs k , rectangular image, k from 1 to 450 (the smaller of the two dimensions)

- Same pattern as the square image - error drops fast at first then flattens, meaning most of the useful information sits in the first few dozen singular values.

3. INFERENCE

3.1 What I learned

- SVD reconstructs better than EVD at the same k .
- Reason: singular values are ranked by actual energy contribution (Eckart-Young theorem says truncated SVD is the best possible rank- k approximation). Eigenvalues of a non-symmetric matrix have no such guarantee.
- Most of an image's information sits in a small number of components. $k=150$ out of 512 already gets SVD very close to the original.
- The shape of the error curve itself tells you that, SVD's smooth drop is expected for an optimal method. EVD's up-and-down curve shows it is not optimal.

3.2 Edge cases I ran into

- Complex eigenvalues in EVD: the image matrix is not symmetric, so `np.linalg.eig` can return complex eigenvalues. They always show up in conjugate pairs. Splitting a pair while picking top- k leaves imaginary leftovers in the reconstruction. Fixed by always pulling in the conjugate partner together, even if k ends up one higher than asked.

- Near-zero singular values: a few eigenvalues of $A_{\text{transpose}} * A$ came out as tiny negative numbers from floating point rounding, which breaks `sqrt()`. Clipped them to zero first.
- k bigger than rank: if k goes past the number of non-zero singular values, reconstruction just equals the original and error stays flat near zero.

4. CONCLUSION

- SVD is the better method for image reconstruction here, both by eye and by the Frobenius numbers.
- This matches the theory - SVD always gives the optimal rank- k approximation, EVD does not.
- Both images only needed a small fraction of their full rank to look close to the original, which is the whole point of low-rank approximation.